Overview sensor location and corresponding dataset:

```
SELECT distinct S.sensor_location_id, dataset_id, dataset_title, position_title
FROM Feature_Uniform_Accelerometer_Gait_Cycle F
INNER JOIN Recording R ON F.Recording_ID = R.Recording_ID
INNER JOIN Sensor_location S ON S.sensor_location_id = F.sensor_location_id
order by dataset_id, sensor_location_id
```

Select amount of male/female persons in dataset with certain ID:

```
SELECT
   COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Male' THEN p.Person_ID END) AS Male_Count,
   COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Female' THEN p.Person_ID END) AS Female_Count
FROM Person p
JOIN Raw_Accelerometer ra ON p.Person_ID = ra.Person_ID
JOIN Recording r ON ra.Recording_ID = r.Recording_ID
WHERE r.Dataset_ID = dataset_id;
```

Select amount of stat. feat. 10 per person of a specific dataset and sensor location:

```
SELECT fuagc.Person_ID, r.dataset_id, fuagc.Sensor_Location_ID,
COUNT(fuagc.Feature_statistical_measure_10_id) AS StatCount
FROM Feature_statistical_measure_10 fuagc
JOIN Recording r ON fuagc.Recording_ID = r.Recording_ID
WHERE r.Dataset_ID = 5 AND fuagc.Sensor_Location_ID = 15
GROUP BY fuagc.Person_ID, r.dataset_id, fuagc.Sensor_Location_ID
ORDER BY StatCount ASC;
```

List each unique sensor used in dataset with certain id:

```
SELECT DISTINCT s.*
FROM Sensor s
JOIN raw_accelerometer ra ON s.sensor_id = ra.sensor_id
JOIN recording r ON ra.recording_id = r.recording_id
WHERE r.dataset_id = dataset_id;
```

Select dataset id – sensor location id pairs with at least 10 persons that have at least 10 gait cycles with usable flag true and male/female count of those:

Both 10s can be replaced with whatever number is of interest

```
SELECT
   f.Sensor_Location_ID,
   r.Dataset_ID,
   COUNT(DISTINCT f.Person_ID) AS Person_Count,
        COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Female' THEN f.Person_ID END) AS
Female_Count,
   COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Male' THEN f.Person_ID END) AS Male_Count
FROM
   Recording r
JOIN
   Feature_Uniform_Accelerometer_Gait_Cycle f ON r.Recording_ID = f.Recording_ID
JOIN
   Person p ON f.Person_ID = p.Person_ID
WHERE
   f.Usable_Flag = TRUE
GROUP BY
   f.Sensor_Location_ID, r.Dataset_ID
HAVING
   COUNT(DISTINCT f.Person_ID) >= 10 AND
COUNT(f.Feature_Uniform_Accelerometer_Gait_Cycle_ID) >= 10
Order By
        r.Dataset_ID
```

Select dataset id – sensor location id pairs with at least 10 persons that have at least 1 statistical_measure_10 value and male/female count of those:

Both 10 and 1 can be replaced with whatever number is of interest

```
SELECT
   f.Sensor_Location_ID,
   r.Dataset_ID,
   COUNT(DISTINCT f.Person_ID) AS Person_Count,
        COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Female' THEN f.Person_ID END) AS
Female_Count,
   COUNT(DISTINCT CASE WHEN p.Sex_Title = 'Male' THEN f.Person_ID END) AS Male_Count
FROM
   Recording r
JOIN
   Feature_Statistical_Measure_10 f ON r.Recording_ID = f.Recording_ID
JOIN
   Person p ON f.Person_ID = p.Person_ID
GROUP BY
   f.Sensor_Location_ID, r.Dataset_ID
HAVING
   COUNT(DISTINCT f.Person_ID) >= 10 AND COUNT(f.Feature_Statistical_Measure_10_ID) >= 1
Order By
        r.Dataset_ID
```

List person id and duration of recording for each recording for a specific dataset_id and activity_id:

Activity_id can be replaced with whatever activity is of interest

Only correctly works if there was only one distinct activity during the recording

```
SELECT
    ra.person_id,
            ra.recording_id,
            ra.sensor_location_id,
    ROUND(r.duration/60000.0, 2) as duration_minutes
  FROM
    raw_accelerometer ra
  JOIN
    recording r ON ra.recording_id = r.recording_id
  WHERE
    r.dataset_id = dataset_id
      AND ra.activity_id = 1
  GROUP BY
    ra.person_id, ra.recording_id, r.duration, ra.sensor_location_id
      ORDER BY
            ra.person_id
```

Get average age and standard deviation of the persons within a certain dataset:

```
SELECT
  ROUND(AVG(p.age)::numeric, 2) AS average_age,
  ROUND(STDDEV(p.age)::numeric, 2) AS stddev_age
FROM Person p
WHERE p.person_id IN (
  SELECT DISTINCT ra.person_id
  FROM raw_accelerometer ra
  JOIN recording r ON ra.recording_id = r.recording_id
  WHERE r.dataset_id = dataset_id);
```

Calculate average walking time per person of a dataset with specific id:

Activity_id can be replaced with whatever activity is of interest

Only correctly works if there was only one distinct activity during the recording

```
WITH person_durations AS (
   SELECT
      ra.person_id,
      COUNT(*) AS entry_count,
      MAX(r.sample_frequency) AS sample_frequency,
      ROUND((COUNT(*) / MAX(r.sample_frequency) / 60)::numeric, 2) AS duration_minutes
   FROM raw_accelerometer ra
   JOIN recording r ON ra.recording_id = r.recording_id
   WHERE r.dataset_id = datset_id
    AND ra.sensor_location_id = (
       SELECT MIN(sensor_location_id)
       FROM raw_accelerometer
       WHERE recording_id IN (
          SELECT recording_id FROM recording WHERE dataset_id = datset_id
       )
    )
   GROUP BY ra.person_id
)
SELECT
   ROUND(AVG(duration_minutes), 2) AS avg_duration_minutes,
   ROUND(STDDEV(duration_minutes), 2) AS stddev_duration_minutes
FROM person_durations;
```

Show which sensor_id belongs to which dataset with internal_coordinate_system_id

for gyroscope and magnetometer swap out raw_accelerometer table for corresponding one together

```
SELECT DISTINCT
   ra.sensor_id,
   r.dataset_title,
        s.internal_coordinate_system_id
FROM raw_accelerometer AS ra
JOIN recording AS r ON ra.recording_id = r.recording_id
JOIN sensor AS s ON ra.sensor_id = s.sensor_id
ORDER BY s.internal_coordinate_system_id,ra.sensor_id, r.dataset_title;
```

Show which Sensor_Location_id belongs to which dataset together with the location_rotation

for gyroscope and magnetometer swap out raw_accelerometer table for corresponding one

```
SELECT DISTINCT
   sl.sensor_location_id,
   sl.position_id,
   sl.position_title,
   sl.rotation,
   r.dataset_title
FROM raw_accelerometer AS ra
JOIN sensor_location AS sl
   ON ra.sensor_location_id = sl.sensor_location_id
JOIN recording AS r
   ON ra.recording_id = r.recording_id
ORDER BY sensor_location_id;
```